

Отчёт по лабораторной работе №7

Основы информационной безопасности

Татьяна Александровна Пономарева

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	8
5	Ответы на контрольные вопросы	9
6	Выводы	11
	Список литературы	12

Список иллюстраций

4.1	Создание файла lab7.py и его содержимое	8
4.2	Запуск lab7.py	8

Список таблиц

1 Цель работы

Освоить на практике применение режима однократного гаммирования

2 Задание

Разработать приложение, позволяющее шифровать и дешифровать данные в режиме однократного кодирования

3 Теоретическое введение

Гаммирование – это наложение (снятие) на открытые (зашифрованные) данные криптографической гаммы, то есть последовательности элементов данных, вырабатываемых с помощью некоторого криптографического алгоритма, для получения зашифрованных (открытых) данных.

С точки зрения теории криптоанализа, метод шифрования случайной однократной равновероятной гаммой той же длины, что и открытый текст, является невскрываемым. Кроме того, даже раскрыв часть сообщения, дешифровщик не сможет хоть сколько-нибудь поправить положение – информация о вскрытом участке гаммы не дает информации об остальных ее частях. «Наложение» гаммы не что иное, как выполнение операции сложения по модулю 2 [1].

4 Выполнение лабораторной работы

Создаем файл lab7.py и пишем код на языке Python для выполнения задания (рис. 4.1).

```
taponomareva@taponomareva:~$ nano lab7.py
GNU nano 8.1 lab7.py
def xor_bytes(data_bytes, key_bytes):
    # Операция XOR между каждым байтом данных и ключа
    return bytes([b ^ k for b, k in zip(data_bytes, key_bytes)])

def task_1():
    print("--- Задание 1: Шифрование ---")
    plaintext = "С Новым Годом, друзья!"
    # Кодировать текст в байты (используем cp1251 или utf-8)
    p_bytes = plaintext.encode('utf-8')

    # Генерируем произвольный ключ той же длины (например, из случайных чисел или фразы)
    # Для простоты возьмем фразу "ЭтоСекретныйКлюч12345" нужной длины
    key_phrase = "ЭтоСекретныйКлюч1234567"
    k_bytes = key_phrase.encode('utf-8')[:len(p_bytes)]

    # Шифруем: C = P XOR K
    c_bytes = xor_bytes(p_bytes, k_bytes)

    print(f"Открытый текст: {plaintext}")
    print(f"Ключ (HEX): {k_bytes.hex().upper()}")
    print(f"Шифротекст (HEX): {c_bytes.hex().upper()}")
    return c_bytes, k_bytes

def task_2(cipher_hex):
    print("\n--- Задание 2: Поиск альтернативного ключа ---")
    c_bytes = bytes.fromhex(cipher_hex)

    # Мы хотим, чтобы тот же шифротекст расшифровался как "Привет от Моллера!!"
    fake_text = "Привет от Моллера!!!"
    f_bytes = fake_text.encode('utf-8')[:len(c_bytes)]

    # Находим фальшивый ключ: K_fake = C XOR P_fake
    k_fake = xor_bytes(c_bytes[:len(f_bytes)], f_bytes)

    print(f"Желаемый ложный текст: {fake_text}")
    print(f"Необходимый для этого ключ (HEX): {k_fake.hex().upper()}")

    # Проверка
    decrypted = xor_bytes(c_bytes[:len(f_bytes)], k_fake).decode('utf-8')
    print(f"Проверка расшифровки: {decrypted}")

# Запуск
cipher_result, key_result = task_1()
task_2(cipher_result.hex())
```

Рисунок 4.1: Создание файла lab7.py и его содержимое

Запускаем файл lab7.py (рис. 4.2).

```
taponomareva@taponomareva:~$ python lab7.py
--- Задание 1: Шифрование ---
Открытый текст: С Новым Годом, друзья!
Ключ (HEX): D0ADD182D08ED0A1D0B5D0BADA180D0B5D182D0BDD18BD0B9D09AD0BBD18ED18721313233343536
Шифротекст (HEX): 000CF1524D6E6E7162645B6A6DA00026013C000901350005FCBA000F000E0004F186E38FE58A17

--- Задание 2: Поиск альтернативного ключа ---
Желаемый ложный текст: Привет от Моллера!!!
Необходимый для этого ключ (HEX): D00320D29D0D6BEC3B2D18AE84D70BEF7831CD095D0BBD0BE2C01D0BAD18ED0B4D0A7C2
Проверка расшифровки: Привет от Моллера!!!
```

Рисунок 4.2: Запуск lab7.py

5 Ответы на контрольные вопросы

1. Гаммирование - это метод шифрования, при котором на открытые данные накладывается (путем операции XOR) последовательность случайных элементов (гамма). Если гамма используется только один раз, имеет длину сообщения и является абсолютно случайной, такой шифр невозможно взломать.
2. Недостатки однократного гаммирования.
 - Длина ключа должна быть равна длине самого сообщения.
 - Сложность безопасной передачи длинных ключей от отправителя к получателю.
 - Трудность генерации по-настоящему случайных (не псевдослучайных) чисел для ключа.
 - Критическое требование: один ключ — одно сообщение. Повторное использование ключа делает систему уязвимой.
3. Преимущества однократного гаммирования.
 - **Абсолютная стойкость:** доказано К. Шенноном, что при соблюдении правил шифр нельзя вскрыть даже при бесконечных вычислительных мощностях.
 - **Высокая скорость:** операция XOR выполняется процессором мгновенно.
 - **Простота реализации:** алгоритм шифрования и дешифрования идентичен.
4. Если ключ короче текста, его приходится использовать повторно (циклически).

Это вносит в шифротекст периодичность, которая позволяет методами статистического анализа вычислить ключ и восстановить исходный текст.

5. В режиме однократного гаммирования используется операция **исключающего ИЛИ** (сложение по модулю 2, **XOR**, $\oplus$). *Особенности:*

- Обратимость: $(A \oplus B) \oplus B = A$.
- Результат равен 0, если операнды одинаковы ($1 \oplus 1 = 0$), и 1, если они разные ($1 \oplus 0 = 1$).
- Равновероятное распределение: если ключ случаен, то результат XOR также будет абсолютно случайным.

6. Шифротекст C получается посимвольным сложением открытого текста P и ключа K по модулю 2:

$$C_i = P_i \oplus K_i$$

7. Поскольку операция XOR обратима, ключ K можно получить, применив XOR к шифротексту C и открытому тексту P :

$$K_i = C_i \oplus P_i$$

8. Согласно К. Шеннону, необходимы три условия:

- **Полная случайность ключа** (равномерное распределение).
- **Равенство длин** ключа и открытого текста.
- **Однократность использования** ключа (после сеанса связи ключ уничтожается).

6 Выводы

В ходе выполнения лабораторной работы применение режима однократного гаммирования было освоено на практике

Список литературы

1. StudFiles. Режим шифрования однократного гаммирования: Программные средства защиты информации [Электронный ресурс]. НГТУ (Новосибирский государственный технический университет). URL: <https://studfile.net/preview/11550163/page:15/> (дата обращения: 15.04.2026).